

Submission Date: 29th April 2026

Create a single zip file with all the files you want to submit. Name the file with the surname of all members of your group (e.g., **x.zip**). Upload your zip in the “Submission” folder of the corresponding week (Exercises → Week 1 - Introduction → Submission).

1 Numerical Questions

1. Receptive Field Calculation

Consider a CNN with the following sequential architecture acting on a 1D input. In the notation below, K denotes kernel size, S stride, P padding, and D dilation factor.

- **Layer 1** (L_1): Standard Convolution ($K=3, S=1, P=1, D=1$),
- **Layer 2** (L_2): Max Pooling ($K=2, S=2$),
- **Layer 3** (L_3): Dilated Convolution ($K=3, S=1, P=1, D=2$).

Tasks:

- Calculate the **effective kernel size** of the dilated convolution in layer L_3 .
- Let R_l denote the receptive field size after layer l . Write the **recurrence relation**, i.e. the recursive update formula, expressing R_l in terms of R_{l-1} , K_l^{eff} , and J_{l-1} , where

$$J_{l-1} = \prod_{j=1}^{l-1} S_j, \quad J_0 = 1.$$

- Using the recurrence relation you found in step (2), compute the scalar **Receptive Field size** for a single neuron in the output of L_3 .
- Does the padding $p=1$ in layers L_1 and L_3 affect the size of the Receptive Field? Briefly justify your answer.

2. Softmax and Logits

Given a logits vector $\mathbf{z} \in \mathbb{R}^K$, the Softmax function is defined as

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Tasks:

- Translation Invariance:** Prove that $\sigma(\mathbf{z}-c) = \sigma(\mathbf{z})$ for any scalar constant c . Briefly explain how this property is utilized to ensure **numerical stability** when implementing Softmax on hardware with limited precision (prevention of floating-point overflow).
- Temperature Limits:** Consider the temperature-scaled Softmax $\sigma(\mathbf{z}/\tau)$.
 - Analytically determine the limiting distribution $\mathbf{y}$ as $\tau \rightarrow \infty$.
 - Analytically determine the limiting distribution $\mathbf{y}$ as $\tau \rightarrow 0^+$, assuming there is a unique maximum in $\mathbf{z}$.
- Entropy Analysis:** Based on your findings in Task 2, does increasing the temperature τ increase or decrease the entropy of the output distribution? Justify your answer.

3. Efficient Architectures (Depthwise Separable Convolutions)

You are designing a lightweight network layer to transform an input tensor of size $H \times W \times C_{in}$ to an output tensor of size $H \times W \times C_{out}$.

Configuration:

- **Input Channels:** $C_{in} = 64$,
- **Output Channels:** $C_{out} = 128$,
- **Spatial Kernel:** $k \times k = 3 \times 3$,
- **Stride:** $s = 1$,
- **Padding:** $p = 1$ (maintaining spatial resolution $H \times W$).

Architectures to Compare:

- **Architecture A (Standard):** A standard 2D Convolution with kernel size 3×3 , stride 1, padding 1.
- **Architecture B (Separable):** A Depthwise Separable Convolution consisting of:
 - (a) **Depthwise Layer:** Kernel 3×3 , stride 1, padding 1.
 - (b) **Pointwise Layer:** Kernel 1×1 , stride 1, padding 0.

Tasks:

- Standard Parameters:** Calculate the total number of learnable weights (P_{std}) for Architecture A (exclude biases).
- Separable Parameters:** Calculate the total number of learnable weights (P_{sep}) for Architecture B (exclude biases).
- Efficiency Ratio:**
 - Calculate the precise ratio $\eta = \frac{P_{sep}}{P_{std}}$ for the specific values given above.
 - Derive the general formula for η in terms of k and C_{out} (showing how C_{in} cancels out).
 - **Concept Check:** Does changing the stride s or padding p affect this parameter count ratio? Briefly explain why or why not.
- Structural Assumption:** What specific assumption does the Depthwise Separable architecture make about the relationship between spatial features (e.g., edges, textures) and channel features (e.g., color, spectral type)?

2 Theoretical Questions

1. Inductive Biases and Connectivity

Deep learning models are distinguished by their “inductive biases” — assumptions about the data distribution that are hard-coded into the network architecture. These assumptions allow models to learn efficiently from limited data, provided the data matches the assumptions.

- Contrast the inductive biases inherent to Multi-Layer Perceptrons (MLPs) versus Convolutional Neural Networks (CNNs). Specifically, define and analyze the roles of “Local Connectivity” and “Weight Sharing” in CNNs. How do these mechanisms enforce translation invariance (or equivariance)?
- Based on these biases, explain why an MLP might fail to generalize on high-dimensional image data compared to a CNN, even if the MLP possesses sufficient parametric capacity (width/depth) to perfectly memorize the training set.
- Analyze the resource implications of these biases. Specifically, how do Local Connectivity and Weight Sharing reduce the memory footprint of a CNN compared to a fully connected MLP? Does Weight Sharing reduce the computational cost (FLOPs) of the forward pass compared to a locally connected layer *without* weight sharing? Explain your answer.

2. Vanishing Gradients in Sequence Models

Recurrent Neural Networks process data sequentially, which effectively transforms them into extremely deep networks when unrolled over time.

- Explain why the vanishing gradient problem is intrinsically linked to the recurrent nature of the network and Backpropagation Through Time (BPTT).
- Consider an RNN with hidden state update $\mathbf{h}^{(k)} = \tanh(\mathbf{W}\mathbf{h}^{(k-1)} + \mathbf{U}\mathbf{x}^{(k)} + \mathbf{b})$. For a loss $\mathcal{L}^{(t)}$ computed at timestep t , the gradient with respect to an earlier state $\mathbf{h}^{(0)}$ contains the Jacobian product:

$$\prod_{k=1}^t \frac{\partial \mathbf{h}^{(k)}}{\partial \mathbf{h}^{(k-1)}} = \prod_{k=1}^t \text{diag}(\tanh'(\mathbf{a}^{(k)})) \mathbf{W},$$

where $\mathbf{a}^{(k)} = \mathbf{W}\mathbf{h}^{(k-1)} + \mathbf{U}\mathbf{x}^{(k)} + \mathbf{b}$. Explain how the repeated multiplication of this factor affects the learning of long-term dependencies. (*Hint:* Think about the magnitude of $\tanh'(\cdot)$ and how the norm of a product of many Jacobian factors behaves. *Note:* The actual total gradient includes a sum over all timesteps; the product above captures the essential difficulty.)

3. Loss Surfaces and Convergence

The geometry of the loss landscape in high-dimensional spaces is often counter-intuitive compared to 2D or 3D visualizations.

- Distinguish between a Local Minimum, a Global Minimum, and a Saddle Point in the context of an optimization surface.
- In high-dimensional optimization (deep learning), which is considered a more common obstacle: getting stuck in a local minimum or slowing down near a saddle point? Justify your answer using the properties of the eigenvalues of the Hessian matrix in high dimensions.
- Explain why Stochastic Gradient Descent (SGD), due to its inherent noise, might effectively escape saddle points where standard full-batch Gradient Descent might stagnate.

4. Normalization Techniques

Training deep networks is difficult because the distribution of each layer's inputs changes during training, as the parameters of the previous layers change (Internal Covariate Shift).

- Explain why Batch Normalization (BN) is considered to have a regularization effect, specifically regarding the stochastic noise introduced by using mini-batch statistics rather than population statistics.
- Compare BN, Layer Normalization (LN), and Instance Normalization (IN). For each of the following scenarios, select the theoretically superior technique and justify why based on the statistical properties of the data:
 - Training with a very small batch size (e.g., 2-4 samples).
 - Training RNNs with variable sequence lengths.
 - Style transfer tasks for artistic images.

5. Adversarial Vulnerability

State-of-the-art models can be easily fooled by imperceptible noise, highlighting a divergence between human perception and model decision boundaries.

- Explain the "Linearity Hypothesis" (Goodfellow et al.) regarding adversarial perturbations. Why does this hypothesis suggest that mechanisms used to solve vanishing gradients (e.g., ReLU, LSTM) paradoxically increase vulnerability to adversarial attacks?
- Analyze the relationship between standard accuracy and adversarial robustness. Why might a model with high accuracy still possess a decision boundary geometry that leaves it vulnerable to imperceptible perturbations?
- Does the existence of adversarial examples invalidate the practical utility of deep learning? Discuss what their existence reveals about the difference between the model's learned decision boundary and the true data manifold.

3 Coding Exercise

In this week's coding exercise, you will train a convolutional neural network for image classification on the CIFAR-10 dataset.

Tracking the performance of your training is a very important part of any Deep Learning project. For this reason, in this course we will use Weights & Biases to do so. If you don't have an account yet, please create one now, as you will need it for the coding exercise.

The provided instructions are for running the code in Google Colab.

- Download the exercise zip *adl_ex1.zip*, and decompress it.
- Upload the notebook *ADL_ex1.ipynb* to Colab.
- Change the runtime to GPU: Runtime → Change runtime type → T4 GPU. If the GPU runtime is not available, you can use the CPU runtime.
- Run the code (Runtime → Run all).
- Follow the instructions after the second cell (`!timeout 5m wandb init`) to link your Weights & Biases account.
- Make sure that all cells run without errors, and that your experiment appears in your Weights & Biases dashboard.

Once the setup is done, answer the following questions:

- By analyzing the graphs (train loss, test loss, test accuracy) of your first training, what do you notice?
- What is “wrong” with the implemented network `Net` defined in the code cell 7?

Now you should run a set of experiments, and draw some conclusions by analyzing the metric graphs.

IMPORTANT: In WandB, each experiment should have its own descriptive name. Therefore, before running each experiment, change the experiment name in cell 4 (`args['experiment_name']`) and restart the Colab session (Runtime → Restart session). In addition to the code, you will need to submit a document that includes, for each of the experiments, the graphs created during the experiment, and your conclusions. You can easily do so by using the “Reports” feature of Weights & Biases, which let you interchange graphs from your experiments and text. For example, for the experiment regarding learning rate, you should provide a graph with the comparison of the accuracy (or loss) for different learning rates, and your conclusions.

The experiments you need to run are the following:

- Modify the `Net` class to fix the issue spotted before. For this experiment, don't use convolutions, only MLP.
- Now modify the `Net` class, and implement a Convolutional Neural Network.
- Change the learning rate. Try at least two lower and two higher rates.
- Change the depth of the network (add/remove convolutional blocks).
- Try different non-linear activation functions for your hidden layers.
- Add data augmentation, see the PyTorch documentation.
- Try at least two additional types of regularization.
- Feel free to experiment more. What is the best test accuracy you were able to achieve?